



ABSCHLUSSPRÄSENTATION

CryptoBroker

Krypto-Trading-Simulator • **CryptoSim**

Risikofreier Einstieg in den Krypto-Handel – echte Live-Kurse, virtuelles Kapital, keine Verluste.

Sara

Anforderungen & QA

Lucas

Backend & DevOps

Julian

Architektur & DB

Jonathan

Frontend & UI

The screenshot shows the 'CryptoSim' web application. At the top, there is a navigation bar with links: Home, Dashboard, Portfolio, Bestenliste, Transaktionen, and Profil. Below the navigation bar, the main heading is 'Alle Coins' with a subtext 'Klicke auf einen Coin für Details.' Below this is a table listing various cryptocurrencies.

Bild	Name	Symbol	Preis	24h %	Market Cap
	Bitcoin	BTC	53.337,00 €	-1.28 %	1.068.675.943.658,00 €
	Ethereum	ETH	1.415,15 €	-2.12 %	170.904.302.834,00 €
	Tether	USDT	0,86 €	+0.15 %	161.578.569.411,00 €
	BNB	BNB	509,84 €	-1.09 %	68.693.703.578,00 €
	USDC	USDC	0,87 €	+0.19 %	64.894.240.737,00 €
	XRP	XRP	0,97 €	-3.35 %	60.092.574.021,00 €
	Solana	SOL	55,66 €	-2.37 %	32.248.117.906,00 €
	TRON	TRX	0,28 €	+0.07 %	26.470.492.876,00 €
	Figure Heloc	FIGR_HELOC	0,89 €	+0.69 %	16.649.519.654,00 €
	Dogecoin	DOGE	0,07 €	-1.74 %	11.232.462.584,00 €
	Hyperliquid	HYPE	48,66 €	-9.17 %	10.817.968.539,00 €

Agenda

01

Vision & Business Case

Sara

02

Features & Live-Demo

Jonathan

03

Tech-Stack & Technologiewechsel

Lucas · Julian

04

Architektur, ADRs & Datenbank

Julian

05

Clean Code & Qualitätssicherung

Lucas · Sara

06

Metriken & CI/CD-Pipeline

Lucas

07

Projektmanagement & Statistik

Lucas

08

Risiken, Retrospektive & Fazit

Julian · Jonathan · Sara

Projektziel & Vision

🎯 Vision

„Kryptowährungen verstehen und Handeln üben - ohne echtes Geld zu riskieren.“

CryptoBroker bildet einen echten Krypto-Markt nach:
Live-Kurse von CoinGecko, ein persönliches Portfolio und
eine Bestenliste – als sichere, intuitive Lernumgebung mit
niedriger Einstiegshürde.



Echte Marktdaten

100 Coins live über die CoinGecko-API



Portfolio-Tracking

Bestände, Wertverlauf & Gewinn/Verlust



Risikofrei

Virtuelles Startkapital, keine Verluste



Gamification

Bestenliste motiviert zum Mitmachen

Business Case



Problem

- Krypto-Einstieg ist riskant und einschüchternd
- Lernen mit echtem Geld kann teuer werden
- Kaum geschützte Übungsumgebungen mit echten Kursen



Lösung

- Web-Simulator mit virtuellem Startkapital
- Echte Live-Kurse statt Fantasiewerten
- Voller Trade-Workflow: Kauf, Verkauf, Portfolio, Verlauf

Zielgruppen



Krypto-Einsteiger

wollen gefahrlos Erfahrung sammeln



Studierende & Lernende

verstehen Marktmechanik praktisch



Hobby-Trader

testen Strategien im Wettbewerb

Funktionsumfang



Login & Registrierung

bcrypt-gehasht, Auto-Register



Markt-Dashboard

100 Coins, Live-Preise & Rang



Kauf & Verkauf

BUY/SELL in €, Menge oder %



Portfolio

Bestände, Wert, Gewinn/Verlust



Kursverlauf

365-Tage-Charts pro Coin



Bestenliste

Gesamtvermögen aller Nutzer



Favoriten

Coins schnell wiederfinden



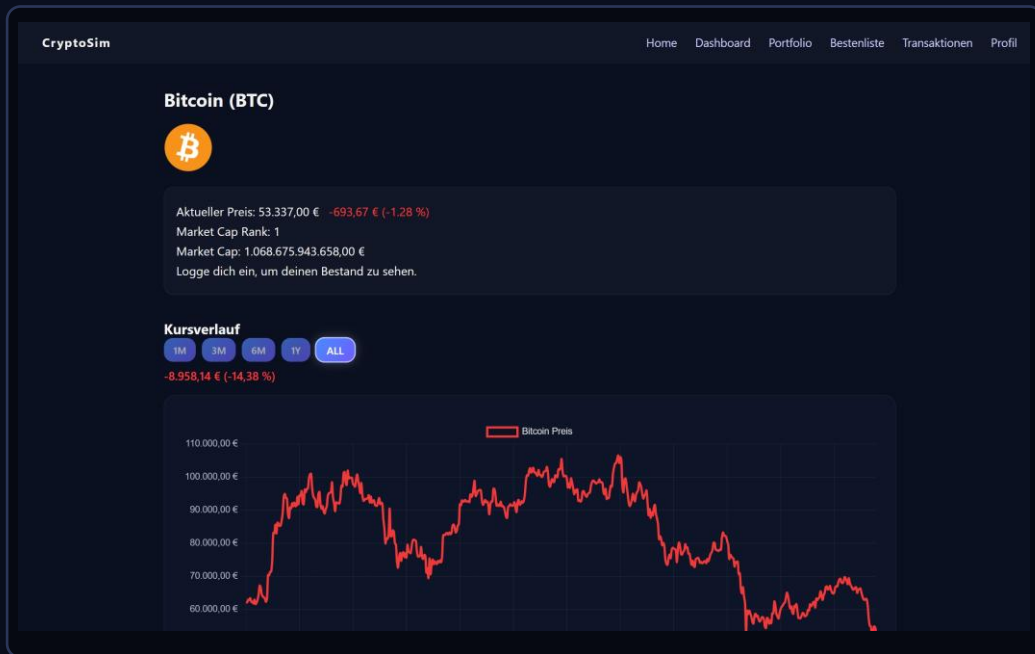
Transaktionen

Vollständige Handelshistorie

Live-Demo

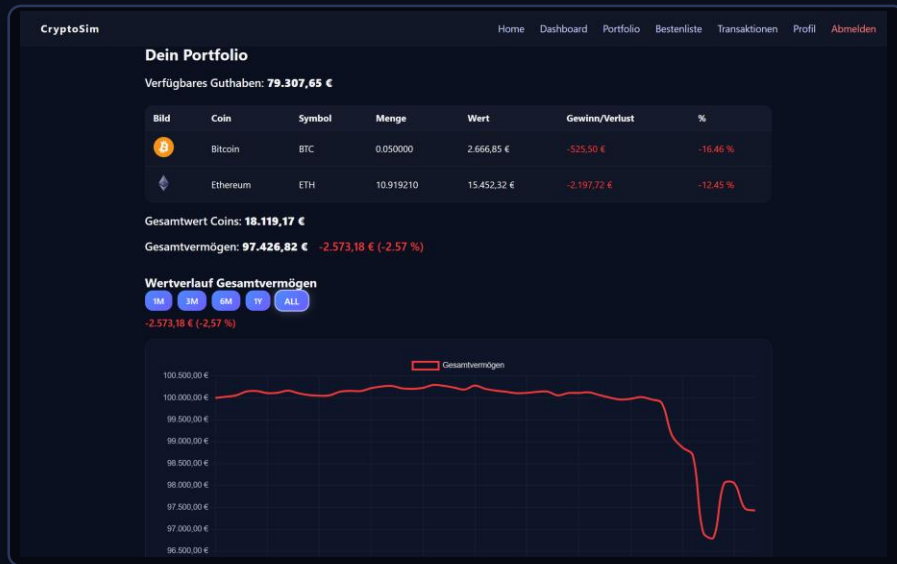
Demo - Ablauf

- 1 Wer handelt selbst Krypto?**
kurze Frage ans Publikum
- 2 Registrieren / Login**
Account anlegen, sicher per bcrypt
- 3 Dashboard erkunden**
Live-Kurse von 100 Coins
- 4 Coin-Detail & Chart**
365-Tage-Kursverlauf ansehen
- 5 Trade ausführen**
Bitcoin kaufen (€ / Menge / %)
- 6 Portfolio & Bestenliste**
Wert, Gewinn/Verlust, Rang



→ Wir zeigen die Anwendung jetzt live im Browser.

Demo-Highlights



Portfolio mit Wertverlauf & Gewinn/Verlust

CryptoSim Home Dashboard Portfolio **Bestenliste** Transaktionen Profil Abmelden

Bestenliste

Gesamtvermögen aller Accounts (Cash + aktueller Coin-Wert).

Rang	Benutzer	Guthaben	Coin-Wert	Gesamtvermögen	Gewinn/Verlust
1	Julian2	99.515,49 €	275,33 €	99.790,82 €	-209,18 € (-0,21 %)
2	Sara (du)	45.000,00 €	53.731,55 €	98.731,55 €	-1.268,45 € (-1,27 %)
3	Jonathan1	91.348,89 €	6.731,59 €	98.080,48 €	-1.919,52 € (-1,92 %)
4	hashed	79.307,65 €	18.119,17 €	97.426,82 €	-2.573,18 € (-2,57 %)
5	Lucas	0,00 €	71.270,29 €	71.270,29 €	-28.729,71 € (-28,73 %)
6	Julian	35.457,62 €	22.721,10 €	58.178,72 €	-41.821,28 € (-41,82 %)

Bestenliste – Gesamtvermögen aller Trader

100

Coins live

365 T

Kurshistorie

3

Eingabemodi

Tech-Stack



Backend

- Python 3
- Flask (Web-Framework)
- Jinja2 (Templates)



Daten & Sicherheit

- SQLite (lokal)
- bcrypt (Password-Hashing)
- REST: CoinGecko-API



Frontend

- HTML5 & CSS3
- Vanilla JavaScript
- Chart.js (Charts)



DevOps & QS

- Docker
- GitHub Actions (CI/CD)
- pytest · pytest-cov · radon

Bewusst leichtgewichtig & teamvertraut – nach Technologiewechsel von Java/JavaFX (siehe nächste Folie).

Technologiewechsel: JavaFX → Flask

VORHER • Java / JavaFX

- ❑ Desktop-Anwendung (JavaFX)
- ❑ Komplexes GUI-Framework
- ❑ Schwierige plattformübergr. Verteilung
- ❑ Keine Web- / REST-Unterstützung
- ❑ Begrenzte Team-Erfahrung



Sprint 2

NACHHER • Python / Flask + Web

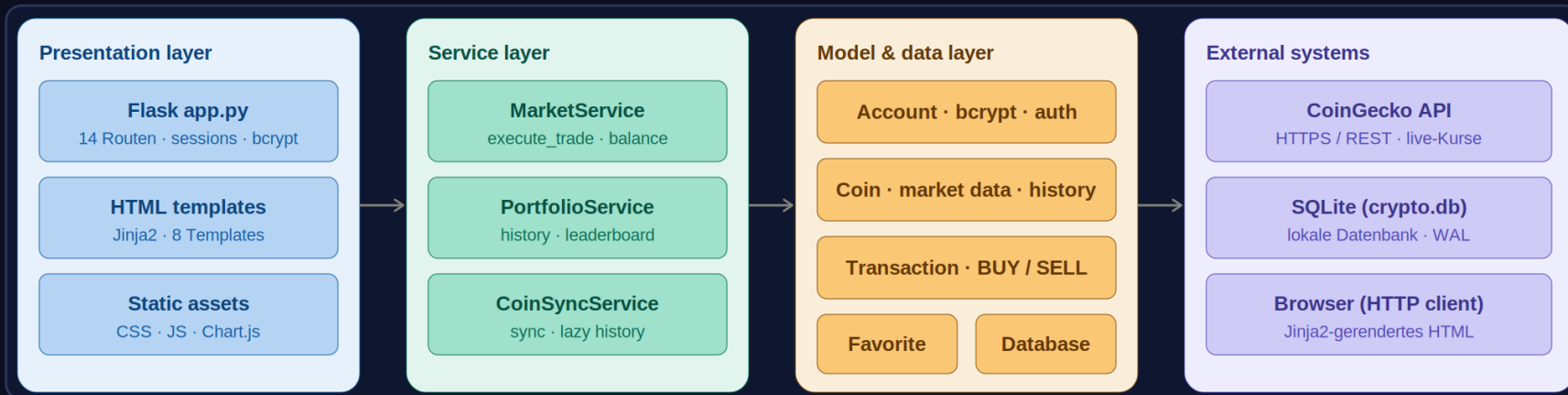
- ✓ Browser-basierte Webanwendung
- ✓ Klare Frontend/Backend-Trennung
- ✓ REST-API-Architektur
- ✓ Einfache Docker-Verteilung & CI/CD
- ✓ Team-vertraute Technologien

+7 h

Mehraufwand
Sprint 2

Key Learning: Technologie-Risiken früh sichtbar machen. Der bewusste Wechsel kostete einmalig 7 Stunden – und sparte über das restliche Projekt ein Vielfaches. Scrum erlaubte das flexible Reagieren.

Architektur: 3 Schichten mit MVC-Elementen



Präsentation / View + Controller (MVC-Elemente)

Flask-Routen (app.py) & Jinja2-Templates, Session-Auth



Service-Schicht

Fat Service: Trade-Logik, Portfolio, API-Sync



Model & Daten

Models kapseln SQLite, zentrale database.py

Architekturentscheidungen (ADRs)

Entscheidung	Begründung
3-Schichten statt Monolith	Trennung der Belange, isoliert testbare Geschäftslogik
Flask statt Django / FastAPI	Minimaler Overhead, teamvertraut, schnelle Entwicklung
SQLite statt PostgreSQL	Keine Server-Infrastruktur, leicht austauschbar (DAL)
bcrypt fürs Hashing	Industrie-Standard, Salt automatisch, sichere Hashes
CoinGecko REST-API	Kostenlos & zuverlässig statt eigener Datenquelle

5 als eigenständige ADRs dokumentiert · arc42 Kapitel 9 (Halbzeit: 2 → jetzt 5)

Datenbank-Design

accounts

id (PK)
name (unique)
pw (bcrypt)
created_at

favorites

acc_id (PK)
coin_id (PK)

transactions

id (PK)
acc_id (FK)
coin_id (FK)
type BUY/SELL
amount · price
timestamp

Event-Sourcing-Prinzip

Guthaben und Bestände werden **nicht gespeichert**, sondern live aus allen Transaktionen berechnet.

coins

id (PK)
symbol · name
current_price
market_cap · rank
image (BLOB)

<coin>_history

timestamp_ms (unique)
price
(je Coin eine Tabelle)

Clean Code & Refactoring

Angewandte Prinzipien

SRP

Eine Funktion, eine Aufgabe

DRY

Keine Duplizierung

IOSP

Integration von Operation
trennen

SoC

Datenhaltung/Logik/View
getrennt

klein

Kleine, sprechende
Funktionen

Beispiel: `execute_trade()` nach IOSP

VORHER

```
def execute_trade(...):  
    # Validierung + DB-Zugriff + Berechnung  
    # vermischt, BUY/SELL-Logik dupliziert
```

NACHHER

```
def execute_trade(self, ...):  
    action = self._normalize_action(action)  
    coin   = self._require_coin(coin_id)  
    self._validate_trade(...)  
    self._save_transaction(...)
```

4+ dokumentierte Refactorings + Design Patterns: Decorator (@login_required) · Service Layer · Event-Sourcing · IOSP (execute_trade) · DRY (Tabellennamen, BUY/SELL) · login_or_register entflochten

Qualitätssicherung: Tests

TEST-COVERAGE

92 %



0 %

Ziel ≥ 80 %

auf services/market_service.py

35

Testfälle grün

33

Unit-Tests

2

Integrationstests

Teststrategie

- Fokus auf die Service-Schicht – dort liegt die kritische Geschäftslogik (Balance, Position, Trade-Validierung, Order-Modi)
- pytest + pytest-cov · 3 Testdateien (test_market_service, test_place_order, test_trade_flow) · isolierte In-Memory-SQLite pro Test
- Bewusst nicht getestet: Flask-Routen, Templates, API-Sync
- Automatische Ausführung bei jedem Push & Pull-Request

Review-Protokoll

Walkthrough-Review

25.05.2026 · 14:00–15:30 Uhr

Geprüft: app.py, market_service.py,
coin_sync_service.py
Rollen

Lucas Moderator

Julian Vorleser

Sara Protokoll

Jonathan Zeitwächter

Methodik: Informaler Walkthrough – Code abschnittsweise vorgelesen, Befunde direkt protokolliert.

Zentrale Befunde → 5 Aufgaben (alle behoben)

- DROP TABLE beim Start → bei API-Fehler leere Coins-Tabelle → INSERT OR REPLACE
- Login-Check in jeder Route dupliziert → zentraler @login_required-Decorator
- Tote while-Schleife in _fetch_coin_history → entfernt
- Portfolio doppelt berechnet → Zwischenergebnis weitergereicht
- Imports & Konstanten an Dateianfang verschoben

Bewährt: sauberes bcrypt-Hashing · inkrementeller API-Sync (nur fehlende Tage) · wiederverwendetes _apply_single_transaction (DRY).

Software-Metriken

2,68

Ø Zyklomat. Kompl. (A)

92 %

Test-Coverage

A

Wartbarkeitsindex

~960

Code-Zeilen (SLOC)

Wartbarkeitsindex pro Modul (höher = besser, ≥ 20 = gut)



Refactoring sichtbar gemacht

`execute_trade()` nach IOSP nur noch **CC 1 (A)**

`/trade`-Route nur noch **B (7)** – 3 Eingabemodi
jetzt in `place_order`; bewusst belassener
Hotspot: `get_leaderboard` C (11).

Gemessen mit `radon (cc · mi · raw)` über 87 Blöcke (ohne Tests) – automatisch in der CI-Pipeline.

CI/CD-Pipeline

Trigger: jeder push und pull_request auf **main** · Plattform: GitHub Actions



Stufe 2 & 3 starten erst nach grünen Tests (needs: tests) – schneller, verlässlicher Feedback-Loop.

Projektmanagement: Scrum

Sprint-Zeitstrahl · Okt 2025 – Jun 2026 · 2 Semester (Halbzeit nach Sem. 3)

Sem 3

JavaFX → Flask, CI/CD,
Halbzeit

S9

Performance &
Architektur

S10

Virt. Guthaben, RMMM

S11

Refactoring, Tests &
Metriken

S12

technisches Review

S13

Leaderboard,
Retrospektive



Scrum-Rahmen

Rollen, iterative Sprints, Backlog-Priorisierung (P0/P1/P2)



Scrum-Board (GitHub)

Für Projekt- und Vorlesungsaufgaben genutzt



Git-Workflow

Feature-Banches & Reviews vor dem Merge (DoD: 2 Devs)



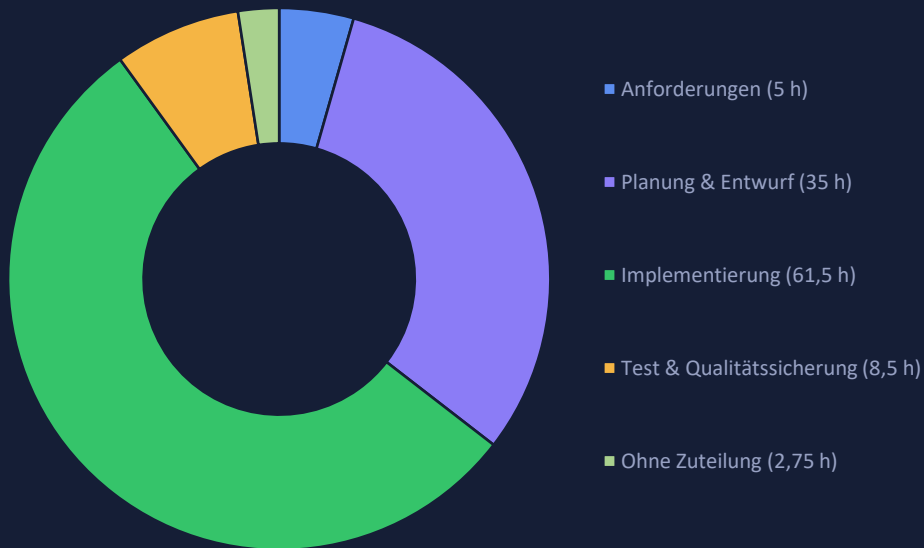
Definition of Done

Kompiliert, ≥ 2 Unit-Tests, dokumentiert, geprüft

54 Items · 52 Done · Anforderungen als User Stories auf dem Scrum-Board

Statistik & Aufwand

Stunden pro Disziplin



Stunden pro Person (Personenstunden)

46,5 h

Lucas

42,5 h

Sara

43 h

Jonathan

47 h

Julian

112,75 h Netto-Aufwand

von 119,5 h geplant — **94 % Auslastung**

Netto-Aufwand = Aufgabenstunden (geteilte Aufgaben einmal gezählt). Summe der Personenstunden: 179 h; die Differenz von 66 h entfällt auf gemeinsame Arbeit (Pair-Programming, Reviews, Walkthrough). Erfasst über das GitHub-Scrum-Board.

Risiken (RMMM)

ID	Beschreibung	W	S	Score	Strategie
T-01	CoinGecko-API-Ausfall / Rate-Limit	3	4	12	Lokales Caching in SQLite, INSERT OR REPLACE
T-02	SQLite Race-Conditions bei Trades	2	5	10	WAL-Modus aktivieren
P-01	Teamausfall (Krankheit / Prüfungsphase)	3	3	9	Pair-Programming, Code-Reviews, Wissen teilen
P-02	Scope Creep – Feature-Wachstum	4	3	12	Strikte Definition of Done pro Sprint
P-03	Fehlende automatisierte Tests	4	3	12	pytest-Framework + CI eingeführt
P-04	Zeitdruck vor Abgabe / Blog-Deadlines	3	2	6	Wöchentliche Sprints, Blogs parallel

 ≤ 8 gering 9–14 mittel ≥ 15 hoch

6 Risiken (2 technisch · 4 Projekt) · vollständige Liste:
docs/RMMM_CryptoBroker.xlsx

Retrospektive & Erkenntnisse



Technologie früh evaluieren

Der JavaFX→Flask-Wechsel kostete 7 h – früher erkannt, hätte er noch weniger gekostet.



Tests & CI früher einführen

Automatisierte Tests hätten Regressionen von Anfang an gefangen.



Reviews lohnen sich

Das gravierende DROP-TABLE-Problem fiel erst beim gemeinsamen Draufschaun auf.



Clean Code zahlt sich aus

Gezielte Refactorings (IOSP, DRY) senkten die Komplexität messbar – `execute_trade` auf A(1) – und machten die Service-Schicht isoliert testbar.

Fazit - worauf wir stolz sind



Vollständiges Produkt

Funktionierender Trading-Simulator mit echten Live-Daten



Saubere Architektur

3 Schichten, 5 begründete ADRs, arc42-Dokumentation



Echte Qualität

35 Tests grün, 92 % Coverage (market_service), Metriken in der Pipeline



Automatisierte Pipeline

Tests, Metriken & Docker-Build bei jedem Push

100

Coins

35

Tests

92 %

Coverage

5

ADRs

112,75h

Aufwand



VIELEN DANK

Fragen?

Wir freuen uns auf eure Fragen zum CryptoBroker.

Ressourcen

Code-Repository

github.com/Lucas-Klein1/CryptoBroker

Doku [arc42](#) · [SRS](#) · [ASR](#) · [Testplan](#) · [Review](#) · [Metriken](#) (im /docs-Verzeichnis)

The screenshot shows the 'CryptoSim' application interface. At the top, there is a navigation bar with links: Home, Dashboard, Portfolio, Bestenliste, and Transaktion. Below this, the 'Transaktionshistorie' section is displayed. It features three filter buttons: 'Alle' (selected), 'Käufe', and 'Verkäufe'. To the right of these buttons are four time range filters: 'Alle', '7 Tage', '1 Monat', and '1 Jahr'. Below the filters is a table with the following columns: Bild, Coin, Typ, Menge, Preis, Gesamt, and Datum. The table contains four rows of transaction data.

Bild	Coin	Typ	Menge	Preis	Gesamt	Datum
	Ethereum	Kauf	6.557807	1.524,90 €	10.000,00 €	04.06.2026 22:59
	Ethereum	Verkauf	1.453801	1.719,63 €	2.500,00 €	30.05.2026 01:58
	Ethereum	Kauf	5.815204	1.719,63 €	10.000,00 €	30.05.2026 01:56
	Bitcoin	Kauf	0.050000	63.847,00 €	3.192,35 €	20.04.2026 11:23